

# Experimentelle Analyse von Algorithmen für fair-k-center mit Ausreißern

Bachelorarbeit

von

Carsten Krollmann

aus

Pirmasens

vorgelegt bei

Lehrstuhl für Algorithmen und Datenstrukturen

Prof. Dr. Melanie Schmidt

Heinrich-Heine-Universität Düsseldorf

Februar 2013

Betreuer:

Dr. Daniel Schmidt

---

# Zusammenfassung

Hier beschreibe ich nun auf einer Seite, welches Problem ich betrachte, wie ich es gelöst habe und was die Hauptergebnisse meiner Arbeit sind.

Laut Prüfungsordnung muss es eine Zusammenfassung geben, die nicht länger als eine Seite ist.

# Inhaltsverzeichnis

|                                                |           |
|------------------------------------------------|-----------|
| <b>List of Algorithms</b>                      | <b>v</b>  |
| <b>1 Einleitung</b>                            | <b>1</b>  |
| 1.1 Motivation . . . . .                       | 1         |
| 1.2 Aufbau der Arbeit . . . . .                | 1         |
| <b>2 Algorithmen</b>                           | <b>2</b>  |
| 2.1 Matching . . . . .                         | 2         |
| 2.1.1 Bipatites Matching . . . . .             | 2         |
| 2.2 Max-Flow Algorithmus . . . . .             | 3         |
| 2.2.1 Flusseigenschaften . . . . .             | 3         |
| 2.3 Gonzalez . . . . .                         | 3         |
| 2.4 Fairletts . . . . .                        | 4         |
| 2.5 Fair-Clustering mit Ausreißern . . . . .   | 5         |
| 2.5.1 Outlier-Punkte . . . . .                 | 5         |
| 2.5.2 Red-Clustering . . . . .                 | 5         |
| 2.5.3 Fast-Anchor-Clustering . . . . .         | 8         |
| <b>3 Aufbereiten der Daten</b>                 | <b>11</b> |
| 3.1 Ursprungsdaten . . . . .                   | 11        |
| 3.1.1 Problem der Datendokumentation . . . . . | 12        |
| 3.2 Cleaning-Pipeline . . . . .                | 12        |
| 3.2.1 Normalisierung . . . . .                 | 12        |
| 3.2.2 Reinigen und Filtern . . . . .           | 12        |
| 3.2.3 Subsampling . . . . .                    | 13        |
| 3.2.4 Format der Datensätze . . . . .          | 13        |

|                                       |           |
|---------------------------------------|-----------|
| <b>4 Experimente</b>                  | <b>14</b> |
| 4.1 Graphen . . . . .                 | 14        |
| <b>5 Auswertung</b>                   | <b>15</b> |
| 5.1 Beschreibung der Bilder . . . . . | 15        |
| 5.2 Güte der Algorithmen . . . . .    | 15        |
| <b>6 Zitierungen</b>                  | <b>16</b> |
| <b>Literatur</b>                      | <b>17</b> |

# List of Algorithms

|   |                                                  |   |
|---|--------------------------------------------------|---|
| 1 | Gonzalez-Algorithmus . . . . .                   | 4 |
| 2 | Farilett-Finder Red Clustering . . . . .         | 7 |
| 3 | Red Clustering Algorihtmus . . . . .             | 8 |
| 4 | Farilett-Finder Fast-Anchor Clustering . . . . . | 9 |

# **Kapitel 1**

## **Einleitung**

### **1.1 Motivation**

Das hier ist die Einleitung. Hier sollte darauf eingegangen werden, was die Motivation der Arbeit ist.

### **1.2 Aufbau der Arbeit**

Gehe kurz darauf ein, was den Leser in den nächsten Abschnitten erwartet. Die Gliederung kann vorab mit der betreuenden Person abgesprochen werden.

# Kapitel 2

## Algorithmen

In der Arbeit wurden einige verschiedene Algorithmen verwendet. Einige davon sind als gegeben vorausgesetzt. Diese sollen hier aufgezählt und erläutert werden

### 2.1 Matching

Ein Matching ist eine Aufteilung eines Graphen in disjunkte 2er Paare, wobei gilt, dass alle Punkte in einem Paar sind, die Paare sich nicht überschneiden und die Knoten eines Paares über eine Kante verbunden sind.

#### 2.1.1 Bipatites Matching

Wird ein Matching in einem Bipatiten Graphen, also einem Graphen welcher in zwei disjunkte Mengen aufgeteilt werden kann, sodass nur Kanten zwischen den zwei Mengen existieren, gesucht, dann ist das ein Bipatites Matching.

## 2.2 Max-Flow Algorithmus

Das Bilden der Fairletts greift auf die Berechnung eines maximalen Flusses in einem Graph-Netzwerk und das daraus resultierende Matching zurück. Dabei haben die Kanten zwischen den Knoten eine Kapazität über welche ein Flusswert fließen kann.

### 2.2.1 Flusseigenschaften

Für einen Fluss, welcher durch ein Flussnetzwerk fließt, müssen immer folgende 3 Eigenschaften gelten:

1. Jede Kante im Netzwerk muss eine nicht negative Kapazität und Fluss haben.
2. *Flusserhaltung*: Die Summe des einfließenden und des ausfließenden Flusses muss bei jedem Knoten mit Ausnahme von Quelle  $s$  und Senke  $t$  immer 0 sein. Bei der Quelle ist die Summe negativ, bei der Senke positiv.
3. *Kapazitätserhaltung*: Der Fluss welcher über eine Kante fließt, darf zu keinem Zeitpunkt die Kapazität dieser Kante überschreiten.

Beim maximalen Fluss versucht man zwischen zwei Knoten, der Quelle  $s$  und der Senke  $t$ , möglichst viel Fluss zu schicken ohne die Flusseigenschaften zu verletzen. Im Folgenden wird das benutzt, um Knoten verschiedener Farbe in einem Bipatiten Graphen zu matchen. Bei der hier verwendeten Implementierung wurde auf die Bibliothek Lemon Library [Lem] zurückgegriffen. Diese bietet die Möglichkeit selbst Graphen aufzubauen und dann den Maximalen Fluss in diesen zu berechnen. Verwendet wurde hier der Preflow Algorithmus.

## 2.3 Gonzalez

Der Algorithmus von Gonzalez [Gon85] berechnet auf einer Punktmenge ein Clustering. Allerdings berücksichtigt er dabei keine Fairness. Dennoch wird er als Teil des fairen Algorithmus hier verwendet. Der Algorithmus ist ein greedy-Algorithmus, welcher  $k$  Cluster sucht. Er hat al-



so als Eingabe eine Menge an Punkten mit einer Metrik ( $\text{dist}$ ) und ein  $k$  welches die Anzahl an gewünschten Clustern angibt. Er beginnt damit, dass man einen beliebigen Punkt als Zentrum festlegt. Anschließend wird der Punkt, welcher am weitesten weg von diesem ersten Zentrum ist, als nächstes Zentrum gewählt. Für das dritte Zentrum wird der Punkt gewählt für den gilt, dass der Abstand zu den anderen Zentren maximal ist. Diese Schritte wiederholt man jetzt so oft bis man  $k$  verschiedene Zentren hat. Zum Schluss wird jeder Punkt dem Zentrum zugewiesen, zu welchem er den geringsten Abstand hat. Zurück gibt der Algorithmus die Zentren, die Knoten mit ihrer Clusterzugehörigkeit und den maximalen Radius, also der größte Abstand von einem Knoten zu seinem Zentrum.

---

**Algorithm 1** Gonzalez-Algorithmus

---

```

1: function GONZALEZ( $P, \text{dist}, k$ )
2:    $Z :=$  leere Zentrenmenge
3:    $d_{\min} :=$  Distanz zu nächstem Zentrum
4:    $z_0$  beliebig aus  $P$ 
5:    $z_0.\text{cluster} = 0$ 
6:   for all  $x \in P$  do
7:      $d_{\min} = \text{dist}(z_0, x)$ 
8:      $x.\text{cluster} = 0$ 
9:   end for
10:  for  $i \in 2, \dots, k$  do
11:     $z_i = \text{argmax}(d_{\min})$ 
12:     $z_i.\text{cluster} = 1$ 
13:    for  $x \in P \setminus Z$  do
14:      if  $\text{dist}(z_i, x) < d_{\min}[x]$  then
15:         $d_{\min}[x] = \text{dist}(z_i, x)$ 
16:         $x.\text{cluster} = i$ 
17:      end if
18:    end for
19:  end for
20:  return  $(Z, P)$ 
21: end function

```

---

▷ in  $P$  wurden Punkte aktualisiert

## 2.4 Fairletts

In dieser Arbeit soll faires-Clustering analysiert werden, jedoch kann der Algorithmus von Gonzalez 1 eine solche Nebenbedingung nicht erfüllen. Wir benutzen hier sogenannte Fairletts. Fairletts sind die kleinstmögliche, faire Clustereinheit, welche die Nebenbedingung erfüllt. In dieser

Analyse verwenden wir nur 1 : 1 Fairletts die aus einem roten und einem blauen Punkt bestehen. Die Grundidee ist, dass wir immer, wenn wir einen Punkt zu einem Cluster zuweisen, auch sein Fairlett-Partner dem selben Zentrum zugewiesen wird. So wird die Fairness nie verletzt [Fas23]. In dieser Arbeit werden zwei Arten der Fairlettbildung betrachtet und ihre Performance und Güte miteinander verglichen.

## 2.5 Fair-Clustering mit Ausreißern

### 2.5.1 Outlier-Punkte

In reellen Daten ist das Feature, welches man fair auf die Cluster verteilen will meist nicht perfekt 1 : 1 verteilt. Damit man trotzdem ein faires Clustering durchführen kann, ist es möglich einige Punkte als Outlier zu bestimmen. Diese Outlier werden dann nicht mehr beim Clustering berücksichtigt und werden dementsprechend auch nicht einem Cluster zugewiesen.

In einem Verhältniss von  $n : n + x$  Punkten werden  $n$  1 : 1 Fairletts gebildet und  $x$  Outlier definiert.

### 2.5.2 Red-Clustering

Als erste Art des Clusterings betrachten wir das Red-Clustering. **Referenz fehlt noch, in Skript steht nur für 1/t**

#### 2.5.2.1 Bilden der Red-Cluster Fairletts 2

Erst gewährleisten wir, dass das kritische Feature, welches weniger, also nur  $n$  mal vorkommt, das rote Feature wird. Das gilt ohne Beschränkung der Allgemeinheit, da wir die Farbe linear einfach tauschen können.

Anschließend bauen wir einen bipatiten Graphen auf in dem die roten  $R \in P$  und blauen  $B \in P$  Punkte jeweils in eine Partition geteilt sind  $R \cap P = \emptyset$ ,  $R \cup B = P$  und  $|R| \leq |B|$ .

Als nächstes berechnen wir ein Matching mit einem Flussnetzwerk um die Fairletts zu bestimmen. Dazu fügen wir Quelle  $s$  und Senke  $t$  in das Netzwerk hinzu und ergänzen Kanten zwischen Quelle

und den roten Knoten bzw. Kanten zwischen blauen Knoten und Senke, welche alle die Kapazität 1 haben. Nun brauchen wir noch die Kanten zwischen den roten und den blauen Punkten dazu „raten“ wir ein  $r_{test}$  und fügen alle Kanten zwischen  $a \in R$  und  $b \in B$  für welche gilt  $dist(a, b) \leq r_{test}$ . Dabei wählen wir das  $r_{test}$  aus der Liste von allen möglichen paarweisen Distanzen  $r_{test} \in \{dist(a, b) | a \in R, b \in B\}$ .

In diesem Netzwerk wird nun der maximale Fluss berechnet und wenn der Flusswert  $f$  gleich der Anzahl an roten Punkten ist  $f = |R|$ , wissen wir, dass alle roten Punkte gematcht wurden. Diese Berechnung führen wir so lange mit verschiedenen  $r_{test} \in \{dist(a, b)\}$  bis wir das kleinste  $r_{test}$  gefunden haben für die das Matching erfolgreich war ( $f = |R|$ ). Um die Anzahl der Vergleiche zu verringern sortieren wir die Menge  $\{dist(a, b) | a \in R, b \in B\}$  aufsteigend und suchen  $r_{opt}$  über die Binäre-Suche.

Um jetzt die Fairletts zueinander zu zuweisen erzeugen wir nochmal das Flussnetzwerk mit den Kanten  $e = (a, b)$  für  $\{dist(a, b) \leq r_{opt} | a \in R, b \in B\}$ . Dann betrachten wir die Kanten zwischen den Partitionen  $R$  und  $B$  und sobald ein Fluss von 1 über die Kante  $e$  fließt, weisen wir  $a$  und  $b$  mit  $e = (a, b)$  die selbe FairlettID zu, was sie zu einem Fairlett macht.

### 2.5.2.2 Finden der Outlier

Um die Ausreißer zu finden müssen wir noch alle blauen Punkte suchen, welche nicht gematcht wurden. Für jeden blauen Punkt  $b$  welcher nicht gematcht wurde gilt, dass der Fluss über die Kante  $e_{t_i} = (b, t)$  0 ist. Also gehen wir durch alle Senke-Kanten  $e_{t_i} = (b_i, t)$  für alle  $i \in [0, |B|]$  und wenn der Flusswert dieser Kante  $e_{t_i}$  0 ist, dann markieren wir  $b$  als Ausreißer. Dieser Punkt wird bei der Zuweisung zu den Zentren ausgelassen

### 2.5.2.3 Berechnen der Roten Zentren 3

Jetzt Clustern wir lediglich die roten Punkten mithilfe des Algorithmus von Gonzalez 1. Dabei können wir sicher sein, dass dabei kein Ausreißer als Zentrum gewählt wird, da aufgrund der Bedingung am Anfang  $|R| \leq |B|$  kein roter Punkt ein Ausreißer ist.

Nach diesem Schritt sind alle roten Punkten einem Cluster zugewiesen.

---

**Algorithm 2** Farilett-Finder Red Clustering

---

```

1: function RED-CLUSTER-FAIRLETT( $P$ ,  $\text{dist}$ )
2:    $R := \{x \mid x \in P \wedge x.\text{color} = \text{RED}\}$ 
3:    $B := \{x \mid x \in P \wedge x.\text{color} = \text{BLUE}\}$ 
4:    $R_{\text{pot}} = \{r = \text{dist}(a, b) \mid a \in R \mid b \in B\}$ 
5:    $G :=$  Graph für Flussnetzwerk
6:   for all  $r \in R_{\text{pot}}$  do ▷ verwende Binäre Suche
7:     Füge in  $G$  alle Punkte aus  $R$  ein
8:     Füge in  $G$  alle Punkte aus  $B$  ein
9:     Füge in  $G$  Quelle  $s$  und Senke  $t$  ein
10:    Alle Kanten  $(s, a)$  und  $(b, t)$  mit Kapazität 1 einfügen
11:    for all  $\{(a, b) \mid a \in R \mid b \in B\}$  do
12:      if  $\text{dist}(a, b) \leq r$  then
13:        Füge Kante  $(a, b)$  in  $G$  mit Kapazität 1 ein
14:      end if
15:    end for
16:    Berechne Maximalen Fluss  $f$  von  $G$ 
17:    Vergleiche mit  $f \geq |R|$  für Binäre-Suche
18:  end for
19:  Baue  $G$  neu auf mit  $r_{\text{opt}}$  wie in der oberen Schleife auf
20:  Berechne den maximalen Fluss  $f$  auf  $G$ 
21:  for all  $e_{\text{main}_i} \in \{(a, b) \mid a \in R \mid b \in B\}$  do
22:    if  $f(e_{\text{main}_i}) = 1$  then
23:      Weise  $e.a$  und  $e.b$  dem gleichen Fairlett zu
24:    end if
25:  end for
26:  for all  $e_t \in \{(b, t) \mid b \in B\}$  do
27:    if  $f(e_t) = 0$  then
28:      Markiere  $b$  als Ausreißer
29:    end if
30:  end for
31: end function

```

---

### 2.5.2.4 Zuweisen der blauen Partner

Zum Schluss gehen wir durch alle blauen Punkte durch und weisen sie dem selben Cluster zu, zu welchem auch ihr roter Fairlett-Partner zugewiesen wurde oder überspringen sie, falls sie Ausreißer sind.

---

#### Algorithm 3 Red Clustering Algorithmus

---

```

1: function RED-CLUSTERING( $P$ ,  $\text{dist}$ ,  $k$ )
2:   Berechne und markiere Fairletts mit RED-CLUSTER-FAIRLETT( $P$ ,  $\text{dist}$ )
3:    $R := \{x | x \in P \wedge x.\text{color} = \text{RED}\}$ 
4:   Berechne Clustering von  $R$  mit GONZALEZ( $R$ ,  $\text{dist}$ ,  $k$ )
5:    $B := \{x | x \in P \wedge x.\text{color} = \text{BLUE}\}$ 
6:   for all  $b \in B$  do
7:     weise  $b$  dem Selben Cluster zu wie sein Fairlett Partner
8:   end for
9: end function

```

---

### 2.5.3 Fast-Anchor-Clustering

Als zweiten Clustering-Algorithmus betrachten wir den Fast-Anchor-Algorithmus [Fas23]. Dieser arbeitet ähnlich wie der Red-Clustering Algorithmus 2.5.2 bildet aber nach einer grundlegend anderen Bedingung die Fairletts

#### 2.5.3.1 Bilden der Anchor-Fairletts 4

Auch wie im Red-Clustering Algorithmus 2.5.2 bauen wir ein Flussnetzwerk zum finden der Fairletts auf. Unterschiedlich ist nur die Bedingung für das hinzufügen der Kanten zwischen  $a \in R$  und  $b \in B$ . Man testet für alle Paare  $(a, b) \in \{a \in R | b \in B\}$  jeden Punkt  $\alpha \in P$  ob  $\alpha$  als Anker in Frage kommt. Dabei wird der Anker so gewählt, das  $\max(\text{dist}(\alpha, a), \text{dist}(\alpha, b))$  minimal ist.

Beim hinzufügen der Kanten zwischen  $a \in R$  und  $b \in B$  prüfen wir jetzt nicht mehr auf die Bedingung  $\text{dist}(a, b) \leq r_{\text{test}}$ , sondern betrachten den Anker also  $\max(\text{dist}(\alpha, a), \text{dist}(\alpha, b)) \leq r_{\text{test}}$  und fügen dementsprechend Kanten mit Kapazität 1 ein.

Die Ausreißer werden genau so wie beim Red-Clustering bestimmt 2.5.2.2

---

**Algorithm 4** Farilett-Finder Fast-Anchor Clustering

---

```

1: function FAST-ANCHOR-FAIRLETT( $P$ ,  $\text{dist}$ )
2:    $R := \{x | x \in P \wedge x.\text{color} = \text{RED}\}$ 
3:    $B := \{x | x \in P \wedge x.\text{color} = \text{BLUE}\}$ 
4:    $R_{\text{pot}} = \{r = \text{dist}(a, b) | a \in R | b \in B\}$ 
5:    $G :=$  Graph für Flussnetzwerk
6:    $A :=$  Ankermatrix  $|R| \times |B|$ 
7:
8:   for all  $(a, b) | a \in R | b \in B$  do                                     ▷ Ankerberechnung
9:     for all  $\alpha \in P$  do
10:      if  $\max(\text{dist}(\alpha, a), \text{dist}(\alpha, b)) \leq A[a, b]$  then
11:         $A[a, b] = \max(\text{dist}(\alpha, a), \text{dist}(\alpha, b))$ 
12:      end if
13:    end for
14:  end for
15:
16:  for all  $r \in R_{\text{pot}}$  do                                               ▷ verwende Binäre Suche
17:    Füge in  $G$  alle Punkte aus  $R$  ein
18:    Füge in  $G$  alle Punkte aus  $B$  ein
19:    Füge in  $G$  Quelle  $s$  und Senke  $t$  ein
20:    Alle Kanten  $(s, a)$  und  $(b, t)$  mit Kapazität 1 einfügen
21:    for all  $\{(a, b) | a \in R | b \in B\}$  do
22:      if  $\max(\text{dist}(A[a, b], a), \text{dist}(A[a, b], b)) \leq r$  then
23:        Füge Kante  $(a, b)$  in  $G$  mit Kapazität 1 ein
24:      end if
25:    end for
26:    Berechne Maximalen Fluss  $f$  von  $G$ 
27:    Vergleiche mit  $f \geq |R|$  für Binäre-Suche
28:  end for
29:  Baue  $G$  neu auf mit  $r_{\text{opt}}$  wie in der oberen Schleife auf
30:  Berechne den maximalen Fluss  $f$  auf  $G$ 
31:  for all  $e_{\text{main}_i} \in \{(a, b) | a \in R | b \in B\}$  do
32:    if  $f(e_{\text{main}_i}) = 1$  then
33:      Weise  $e.a$  und  $e.b$  dem gleichen Fairlett zu
34:    end if
35:  end for
36:  for all  $e_t \in \{(b, t) | b \in B\}$  do
37:    if  $f(e_t) = 0$  then
38:      Markiere  $b$  als Ausreißer
39:    end if
40:  end for
41: end function

```

---

### **2.5.3.2 Center-Aware Problem**

# Kapitel 3

## Aufbereiten der Daten

Um vergleichbare Werte zu erhalten, haben wir uns an einem Paper orientiert, welches ebenfalls probiert hat, Fair-k-Center zu lösen [Chi+18]. Für die Vergleichbarkeit haben wir auch die gleichen Daten verwendet.

### 3.1 Ursprungsdaten

In dem Chierichetti-Paper [Chi+18] wurden 3 unterschiedliche Datensätze verwendet. Diese sind:

- Daten eines US-Zensus [BK96]
- Daten von Telefongesprächen einer Bank [MRC12]
- Einen Diabetes-Datensatz, wobei der im Paper [Chi+18] verlinkte Datensatz [Kah] nicht der richtige Datensatz sein kann (Er enthält andere Felder als die im Paper genannten). Nachforschungen haben gezeigt, dass sie in Wirklichkeit einen anderen Diabetes-Datensatz [Clo+14] benutzt haben.



### 3.1.1 Problem der Datendokumentation

Ein Problem der Daten war, dass sie einerseits nicht in einem normalisierten CSV Format vorhanden waren und andererseits die Autoren im Paper [Chi+18] Subsamples benutzt, jedoch nicht explizit gesagt haben, welche Datenpunkte genommen und welche Features der Daten verwendet wurden. Bei Letzterem wurden für jeden Datensatz ein paar Features genannt. Genau diese haben wir dann auch verwendet. Das Problem der Normalisierung und des Subsamplings sind wir dann selbst angegangen.

## 3.2 Cleaning-Pipeline

Um die Daten verwendbar zu machen, brauchen wir einige Schritte. Dafür verwenden wir ein Python-Skript, welches einerseits vollautomatisch, andererseits aber auch deterministisch und konfigurierbar die Daten aufbereitet. Auch werden die Daten nach der Normalisierung, der Reinigung und dem Zerteilen jeweils abgespeichert, damit man ggf. mit diesen Daten auch weiterarbeiten kann, ohne sie nochmal neu erzeugen zu müssen.

### 3.2.1 Normalisierung

Als erstes mussten die Daten in ein einheitliches CSV-Format übertragen werden. Das war zwar bei dem Diabetes-Datensatz [Clo+14] nicht notwendig, bei den anderen zwei [BK96] [MRC12] mussten ein paar Zeichen ersetzt werden und die Zensus-Daten brauchten Überschriften.

### 3.2.2 Reinigen und Filtern

Anschließend werden die Daten für unser Clustering weiterverarbeitet. Dazu werden alle Spalten entfernt, die wir nicht brauchen, sodass jeder Datensatz nur aus einigen numerischen Werten und dem Critical-Feature besteht. Gleichzeitig wird auch eine Grundanalyse der Critical-Features (Verhältnisse und mergen von Kategorien die das gleiche bedeuten) durchgeführt. Das Critical-Feature steht immer in der ersten Spalte. Ganz zum Schluss werden alle Daten auf ein Intervall von [-

1, 1] gemappt. So haben Daten mit unterschiedlichen Skalierungen nicht unterschiedlichen große Auswirkung auf das Clustering bzw. die berechneten Distanzen.

### 3.2.3 Subsampling

Da es um Faires-Clustering geht, sind die Versuche stark abhängig von der Verteilung der Ursprungsdaten. Um zu vermeiden, ein sehr ungünstiges Batch zu bekommen, werden alle Daten in 20 disjunkte Subsamples zerteilt, welche die selbe Größe haben wie sie im Paper [Chi+18] genannt werden. In der Analyse werden dann alle Subsamples getestet und dann der Durchschnitt über die Performance gebildet.

### 3.2.4 Format der Datensätze

Hier sollen jetzt nochmal klar die Datensätze dokumentiert werden. Von jedem Datensatz wurden jeweils 20 Disjunkte Subsamples erzeugt.

Tabelle 3.1: Datenformat

| Datensatz              | Critical-Feature                                                                         | Verhältnis des Critical-Feature                                                                                                        | Verwendete numerische Feature                           | Sub-sample-Größe |
|------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|------------------|
| Zensusdaten [BK96]     | sex (male = 0; female = 1)                                                               | Male 21790 / Female 10771 $\approx 2.02$                                                                                               | age, fnlwtg, education-num, capital-gain, hours-perweek | 600              |
| Bankdaten [MRC12]      | bin-marital (selbsterzeugt aus Spalte "marital") (single = 0; divorced = 0; married = 1) | es gibt single, married und divorced. Zähle single und divorced zusammen. Verhältnis: 27214 married / 17997 not-married $\approx 1.51$ | age, balance, duration of call                          | 1000             |
| Diabetesdaten [Clo+14] | sex (male = 0; female = 1)                                                               | Verhältnis Male 47055 / Female 54708 $\approx 0.86$                                                                                    | age, time-in-hospital                                   | 1000             |

# Kapitel 4

## Experimente

- Was genau hab ich jetzt berechnet
- Welche Daten habe ich explizit verwendet (kurz)
- Diagramm der Testing-Pipeline
- Umgebung (vielleicht technische Daten)
- Graphen aus Python

### 4.1 Graphen

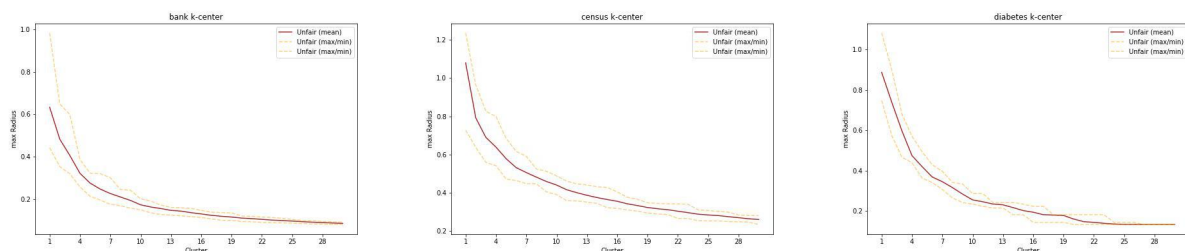


Abbildung 4.1: Kosten für Gonzalez bis zu 30 Cluster

# Kapitel 5

## Auswertung

Was habe ich aus den Experimenten gelernt

### 5.1 Beschreibung der Bilder

### 5.2 Güte der Algorithmen

Konvergieren die Algorithmen gut? Wie groß ist der Kostenunterschied? Wie groß ist der Unterschied zwischen den Datensätzen?

# Kapitel 6

## Zitierungen

Einfach eine Datei, in der alles Zitiert wird, damit ich mir mein Literaturverzeichnis anschauen kann

Noch ein wenig Mehr Zitieren:

1. Chierichetti-Paper [Chi+18]
2. Zensusdaten [BK96]
3. Bankdaten [MRC12]
4. Diabetes (Falsch) [Kah]
5. Diabetes (Richtig) [Clo+14]
6. BA von Irina [Fas23]
7. Vorlesungsskript Clustering [Sch23]
8. Lemon Library [Lem]

# Literatur

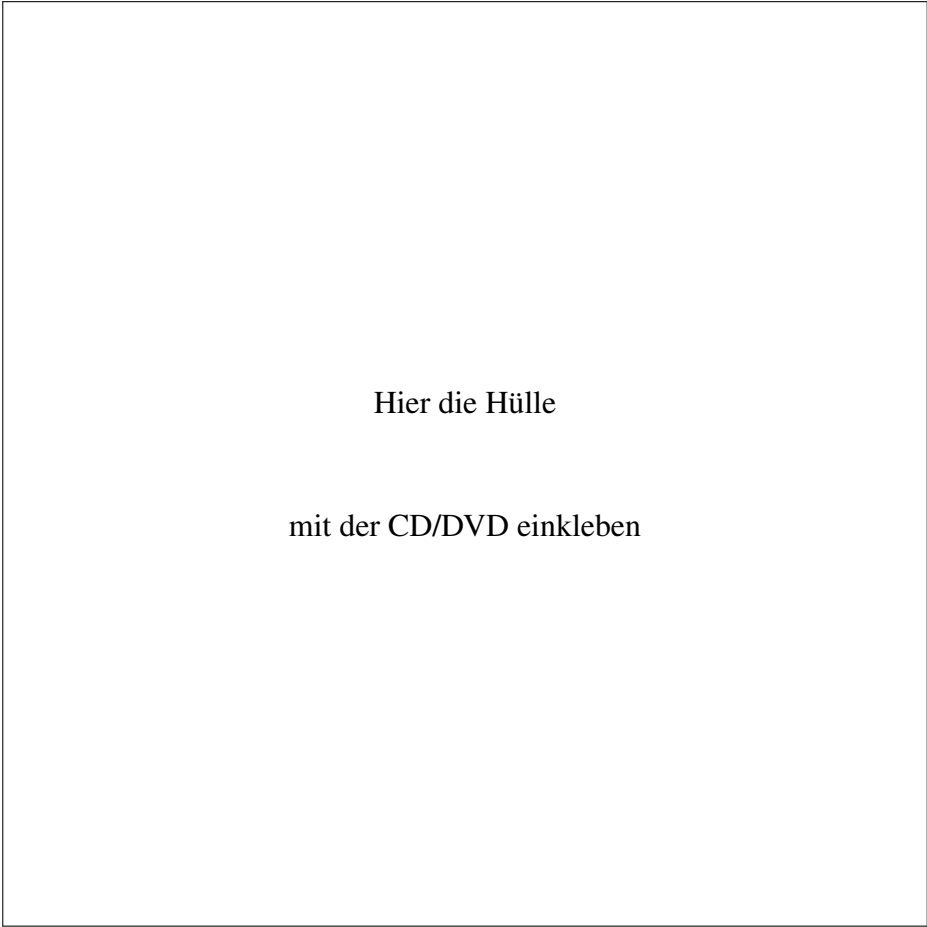
- [BK96] Barry Becker und Ronny Kohavi. *Adult*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5XW20>. 1996.
- [Chi+18] Flavio Chierichetti u. a. *Fair Clustering Through Fairlets*. 2018. arXiv: 1802.05733 [cs.LG].
- [Clo+14] John Clore u. a. *Diabetes 130-US hospitals for years 1999-2008*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5230J>. 2014.
- [Fas23] Irina Fast. „Fair k-Center Clustering mit Ausreißern“. Masterarbeit. Düsseldorf, NRW, Germany: Heinrich Heine University, Dez. 2023.
- [Gon85] Teofilo F. Gonzalez. „Clustering to minimize the maximum intercluster distance“. In: *Theoretical Computer Science* 38 (1985), S. 293–306. ISSN: 0304-3975. DOI: [https://doi.org/10.1016/0304-3975\(85\)90224-5](https://doi.org/10.1016/0304-3975(85)90224-5). URL: <https://www.sciencedirect.com/science/article/pii/0304397585902245>.
- [Kah] Michael Kahn. *Diabetes*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5T59G>.
- [Lem] A Jüttner, B Dezsö und P Kovács. *Lemon Library - open source graph library*. <https://lemon.cs.elte.hu/trac/lemon>.
- [MRC12] S. Moro, P. Rita und P. Cortez. *Bank Marketing*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5K306>. 2012.
- [Sch23] Melanie Schmidt. „Kombinatorische Algorithmen für Clusteringprobleme - Vorlesungsskript“. 2023.

# **Ehrenwörtliche Erklärung**

Hiermit versichere ich, die vorliegende Bachelorarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben. Alle Stellen, die aus den Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Düsseldorf, 29. Februar 2013

Carsten Krollmann



Hier die Hülle  
mit der CD/DVD einkleben

**Diese CD enthält:**

- eine *pdf*-Version der vorliegenden Bachelorarbeit
- die  $\text{\LaTeX}$ - und Grafik-Quelldateien der vorliegenden Bachelorarbeit samt aller verwendeten Skripte
- **[anpassen]** die Quelldateien der im Rahmen der Bachelorarbeit erstellten Software XYZ
- **[anpassen]** den zur Auswertung verwendeten Datensatz
- die Websites der verwendeten Internetquellen